MainActivity.java

Esta clase pertenece al paquete ui/auth y es el punto de entrada principal de la aplicación. Su objetivo es gestionar el inicio de sesión de los usuarios, permitiendo validar sus credenciales a través de Firebase Auth y dirigir a cada persona a su panel correspondiente (Administrador, Guardia o Residente) según el rol detectado. Además, incluye lógica de persistencia para que, si ya habías iniciado sesión antes, no tengas que volver a escribir tus datos, y verifica que tu cuenta esté activa y tu correo verificado, contribuyendo a mantener la seguridad y el acceso controlado a todas las funciones del condominio desde el primer momento.

CODIGO:

```
public class MainActivity extends AppCompatActivity implements
View.OnClickListener {

    private EditText editTextEmail, editTextPassword;

    private MaterialButtonToggleGroup roleToggleButtonGroup;

    private FirebaseAuth mAuth;

    private FirebaseFirestore db;

    @Override

    protected void onCreate(Bundle savedInstanceState) {

        super.onCreate(savedInstanceState);

        setContentView(R.layout.activity_main);

        mAuth = FirebaseAuth.getInstance();

        db = FirebaseFirestore.getInstance();


        findViewById(R.id.buttonLogin).setOnClickListener(this);

        findViewById(R.id.textViewRegister).setOnClickListener(this);

        if (mAuth.getCurrentUser() != null &&
mAuth.getCurrentUser().isEmailVerified()) {

            checkUserRoleAndRedirect(mAuth.getCurrentUser().getUid(), null);

        }

    }


    private void loginUser() {

        String email = editTextEmail.getText().toString().trim();
```

```
        String pass = editTextPassword.getText().toString().trim();
        int checkedId = roleToggleButtonGroup.getCheckedButtonId();
        String selectedRole = (checkedId == R.id.btnRoleAdmin) ? "ADMIN" :
                            (checkedId == R.id.btnRoleGuard) ? "GUARD" : "USER";
        mAuth.signInWithEmailAndPassword(email,
pass).addOnCompleteListener(task -> {
            if (task.isSuccessful()) checkUserRoleAndRedirect(mAuth.getUid(),
selectedRole);
            else Toast.makeText(this, "Credenciales incorrectas",
Toast.LENGTH_SHORT).show();
        });
    }
    private void checkUserRoleAndRedirect(String uid, String selectedRole) {
        db.collection("usuarios").document(uid).get().addOnSuccessListener(doc -
> {
            String role = (selectedRole != null) ? selectedRole : doc.getString("role");
            Intent intent;
            if ("ADMIN".equals(role)) intent = new Intent(this,
AdminDashboardActivity.class);
            else if ("GUARD".equals(role)) intent = new Intent(this,
GuardDashboardActivity.class);
            else intent = new Intent(this, ProfileActivity.class);
            startActivity(intent);
            finish();
        });
    }
    @Override
    public void onClick(View v) {
        if (v.getId() == R.id.buttonLogin) loginUser();
        else if (v.getId() == R.id.textViewRegister) startActivity(new Intent(this,
RegisterActivity.class));
    }
}
```

RegisterActivity.java

Esta clase pertenece al paquete ui/auth y representa el primer paso del proceso de registro para nuevos residentes. Su objetivo es recolectar y validar la información personal básica (nombre, correo, teléfono y dirección) antes de proceder a la creación de la cuenta. La clase asegura que el correo tenga un formato válido y que ningún campo quede vacío, funcionando como un filtro de calidad de datos que garantiza que la base de datos del condominio contenga información fidedigna desde el inicio.

CODIGOS:

```
public class RegisterActivity extends AppCompatActivity {

    private EditText editTextFullName, editTextEmail, editTextPhone,
editTextAddress;


    @Override

    protected void onCreate(Bundle savedInstanceState) {

        super.onCreate(savedInstanceState);

        setContentView(R.layout.activity_register);

        editTextFullName = findViewById(R.id.editTextFullName);

        editTextEmail = findViewById(R.id.editTextEmail);

        editTextPhone = findViewById(R.id.editTextPhone);

        editTextAddress = findViewById(R.id.editTextAddress);


        findViewById(R.id.backButton).setOnClickListener(v -> finish());

        findViewById(R.id.buttonNext).setOnClickListener(v ->
goToSetPassword());

    }

    private void goToSetPassword() {

        String name = editTextFullName.getText().toString().trim();

        String email = editTextEmail.getText().toString().trim();

        String phone = editTextPhone.getText().toString().trim();

        String addr = editTextAddress.getText().toString().trim();


        // Validaciones de campos vacíos y formato de email
```

```
        if (name.isEmpty() || email.isEmpty() ||
!Patterns.EMAIL_ADDRESS.matcher(email).matches()) {
            Toast.makeText(this, "Datos inválidos", Toast.LENGTH_SHORT).show();
            return;
        }

        // Empaquetado de datos para la siguiente actividad
        Intent intent = new Intent(this, SetPasswordActivity.class);
        intent.putExtra("fullName", name);
        intent.putExtra("email", email);
        intent.putExtra("phone", phone);
        intent.putExtra("address", addr);
        startActivity(intent);
    }
}
```

SetPasswordActivity.java

Esta clase pertenece al paquete ui/auth y es la pieza final del proceso de registro. Su objetivo es asegurar la cuenta del usuario mediante una contraseña y sincronizar toda la información recolectada con los servicios de Firebase. La clase realiza una operación crítica en tres pasos: crea la credencial de acceso en Firebase Authentication, guarda el perfil detallado en Cloud Firestore y dispara un correo de verificación. Esto garantiza que cada residente tenga una identidad digital protegida y que sus datos estén listos para ser validados por la administración.

CODIGO:

```
public class SetPasswordActivity extends AppCompatActivity {
    private EditText editTextPassword, editTextConfirmPassword;
    private FirebaseAuth mAuth;
    private FirebaseFirestore db;
    private String fullName, email, phone, address;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_set_password);
        mAuth = FirebaseAuth.getInstance();
        db = FirebaseFirestore.getInstance();
        fullName = getIntent().getStringExtra("fullName");
        email = getIntent().getStringExtra("email");
        phone = getIntent().getStringExtra("phone");
        address = getIntent().getStringExtra("address");
        findViewById(R.id.buttonRegister).setOnClickListener(v -> registerUser());
        findViewById(R.id.backButton).setOnClickListener(v -> finish());
    }
    private void registerUser() {
        String pass = editTextPassword.getText().toString().trim();
        if (pass.length() < 6) return; // Validación básica


        // 1. Crear usuario en Auth
```

```
        mAuth.createUserWithEmailAndPassword(email,
pass).addOnCompleteListener(task -> {
            if (task.isSuccessful()) saveUserData(mAuth.getCurrentUser());
            else showErrorDialog(task.getException().getMessage());
        });
    }
    private void saveUserData(FirebaseUser firebaseUser) {
        User user = new User(fullName, email, phone, address);
        // 2. Guardar perfil en Firestore usando el UID de Auth
        db.collection("usuarios").document(firebaseUser.getUid()).set(user)
            .addOnSuccessListener(aVoid -> {
                // 3. Enviar correo y navegar
                firebaseUser.sendEmailVerification();
                startActivity(new Intent(this, VerifyEmailActivity.class));
                finish();
            });
    }
}
```

VerifyEmailActivity.java

Esta clase pertenece al paquete ui/auth y es la encargada de asegurar que el correo electrónico proporcionado por el usuario sea real y le pertenezca. Su objetivo es pausar el flujo de registro hasta que el usuario confirme su identidad mediante el enlace enviado a su bandeja de entrada. Lo más destacado de esta actividad es el uso de un Handler que verifica automáticamente el estado del usuario cada 3 segundos (reload()), permitiendo que la aplicación detecte la validación en tiempo real y redirija al usuario a la pantalla de espera de activación sin que este tenga que presionar ningún botón adicional.

CODIGO:

```
public class VerifyEmailActivity extends AppCompatActivity {

    private FirebaseAuth mAuth;

    private Handler handler;

    private Runnable checkEmailVerifiedRunnable;

    @Override

    protected void onCreate(Bundle savedInstanceState) {

        super.onCreate(savedInstanceState);

        setContentView(R.layout.activity_verify_email);

        mAuth = FirebaseAuth.getInstance();

        findViewById(R.id.buttonResendEmail).setOnClickListener(v ->
resendEmail());

        handler = new Handler();

        checkEmailVerifiedRunnable = new Runnable() {

            @Override

            public void run() {

                checkIfEmailIsVerified();

                handler.postDelayed(this, 3000);

            }

        };

    }

    private void checkIfEmailIsVerified() {

        FirebaseUser user = mAuth.getCurrentUser();
```

```
        if (user != null) {
            user.reload().addOnCompleteListener(task -> {
                if (user.isEmailVerified()) {
                    handler.removeCallbacks(checkEmailVerifiedRunnable);
                    startActivity(new Intent(this, PendingActivationActivity.class));
                    finish();
                }
            });
        }
    }
    private void resendEmail() {
        FirebaseUser user = mAuth.getCurrentUser();
        if (user != null) user.sendEmailVerification();
    }
}
```

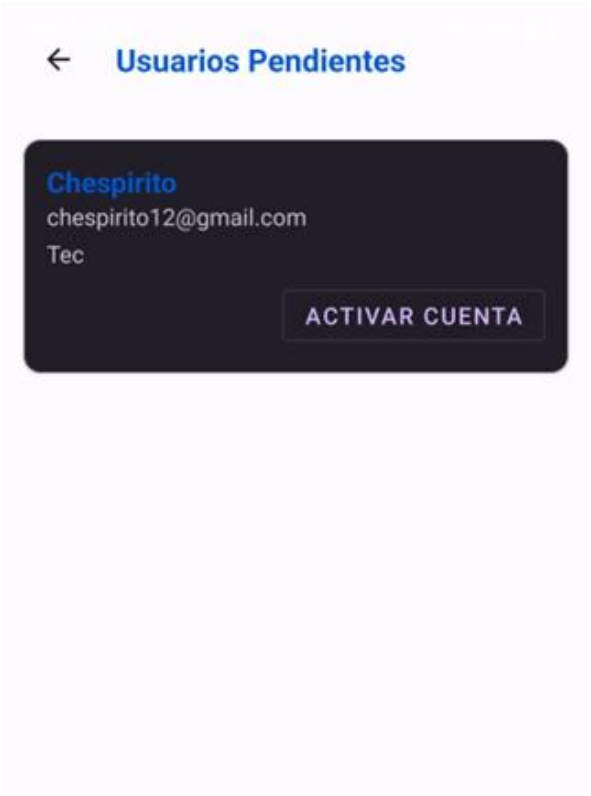
PendingUsersActivity.java

Esta clase pertenece al paquete ui/admin y tiene la tarea de mostrar a los nuevos usuarios que están esperando ser aprobados para entrar al sistema. Su objetivo es permitir al administrador revisar una lista de perfiles con estatus "pendiente", obteniendo la información directamente de Firestore para que nadie use la app sin permiso previo. Esto contribuye a mejorar la seguridad del condominio, asegurando que solo los residentes reales sean activados por el personal administrativo tras validar sus datos.

CODIGO:

```
public class PendingUsersActivity extends AppCompatActivity {
    private RecyclerView recyclerView;
    private UserAdapter adapter;
    private List<User> userList;
    private List<String> userIdList;
    private FirebaseFirestore db;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_pending_users);
        db = FirebaseFirestore.getInstance();
        recyclerView = findViewById(R.id.recyclerViewPendingUsers);
        userList = new ArrayList<>();
        userIdList = new ArrayList<>();
        adapter = new UserAdapter(userList, userIdList);
        recyclerView.setLayoutManager(new LinearLayoutManager(this));
        recyclerView.setAdapter(adapter);
        findViewById(R.id.buttonBack).setOnClickListener(v -> finish());
        loadPendingUsers();
    }
    private void loadPendingUsers() {
        db.collection("usuarios")
                .whereEqualTo("status", "pending")
```

```
            .get()
            .addOnSuccessListener(queryDocumentSnapshots -> {
                userList.clear();
                userIdList.clear();
                for (DocumentSnapshot document : queryDocumentSnapshots) {
                    userList.add(document.toObject(User.class));
                    userIdList.add(document.getId());
                }
                adapter.notifyDataSetChanged();
            });
    }
}
```

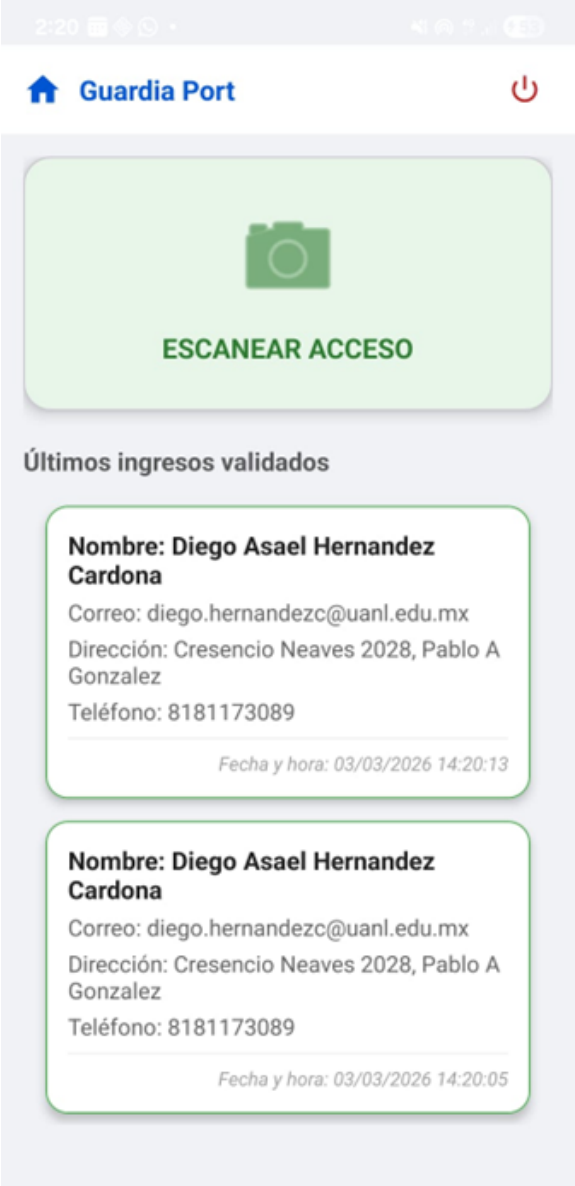


GuardDashboardActivity.java

Esta clase pertenece al paquete ui/tecnico (o personal de seguridad) y funciona como el centro de operaciones para los guardias del condominio. Su objetivo principal es permitir el escaneo de códigos QR para validar el ingreso de personas, actualizando en tiempo real el estado del acceso en la base de datos y notificando mediante avisos en el celular cuando alguien ha sido autorizado. Además, incluye herramientas de supervisión como un historial de ingresos filtrable por fecha y un buscador por correo electrónico, lo que contribuye a mantener un control estricto y una organización eficiente de las entradas y salidas en las instalaciones.

CODIGO:

```
public class GuardDashboardActivity extends AppCompatActivity {

    private FirebaseFirestore db;

    private RecyclerView recyclerViewScanned;

    private ScannedAccessAdapter adapter;

    private List<Map<String, String>> scannedList, fullList;

    private String selectedFilterDate;

    @Override

    protected void onCreate(Bundle savedInstanceState) {

        super.onCreate(savedInstanceState);

        setContentView(R.layout.activity_guard_dashboard);

        db = FirebaseFirestore.getInstance();

        recyclerViewScanned = findViewById(R.id.recyclerViewScanned);

        adapter = new ScannedAccessAdapter(scannedList);

        recyclerViewScanned.setAdapter(adapter);

        findViewById(R.id.cardScanQR).setOnClickListener(v ->
startQRScanner());

        findViewById(R.id.cardSecurityChat).setOnClickListener(v ->
startActivity(new Intent(this, ChatActivity.class)));

        findViewById(R.id.buttonFilterDate).setOnClickListener(v ->
showDatePicker());

        loadHistoryByDate(new SimpleDateFormat("yyyy-MM-dd",
Locale.getDefault()).format(new Date()));

    }
```

```
    private void startQRScanner() {
        IntentIntegrator integrator = new IntentIntegrator(this);
        integrator.setDesiredBarcodeFormats(IntentIntegrator.QR_CODE);
        integrator.setPrompt("Escaneando código de acceso...");
        integrator.initiateScan();
    }
    @Override
    protected void onActivityResult(int requestCode, int resultCode, Intent data) {
        IntentResult result = IntentIntegrator.parseActivityResult(requestCode,
resultCode, data);
        if (result != null && result.getContents() != null) {
            validateScannedQR(result.getContents());
        }
    }
    private void validateScannedQR(String contents) {
        // Separa UID y KEY del QR y busca en Firestore
        String[] parts = contents.split("\\|");
        String userId = parts[0].replace("UID:", "");
        String key = parts[1].replace("KEY:", "");

        db.collection("registro_de_entrada")
            .whereEqualTo("userId", userId).whereEqualTo("key",
key).whereEqualTo("status", "GENERADO")
            .get().addOnSuccessListener(snapshots -> {
                if (!snapshots.isEmpty()) {
                    snapshots.getDocuments().get(0).getReference().update("status",
"AUTORIZADO")
                        .addOnSuccessListener(aVoid -> {
                            showNotification("Acceso Autorizado", "Usuario ingresado.");
                            loadHistoryByDate(selectedFilterDate);
                        });
                }
```

```
        });
    }
    private void loadHistoryByDate(String date) {
        db.collection("registro_de_entrada").whereEqualTo("date",
date).whereEqualTo("status", "AUTORIZADO")
            .get().addOnSuccessListener(query -> {
                scannedList.clear();
                for (DocumentSnapshot doc : query) fetchAndAddToList(doc);
            });
    }
}
```